TECHNICAL DESIGN DOCUMENT (TDD)

B2B Distributor Order Management System & Vernacular Order Engine

Client: Company A, Jabalpur (4 Packaged Food & Home Care Brands, 1,800 Kiranas, 18 Salesmen)
Engineering Stack: Next.js 16 (App Router), TypeScript, Neon PostgreSQL, Drizzle ORM, Vitest
Operational Status: Production Ready • Version 3.2 • 258 Tests (100% Passing) • 97.4% Scorer Precision
Repository: https://github.com/manva-niso/Distributor-Order-Management-System.git

1. Executive Summary & Problem Scope

Company A is an established FMCG distributor in Jabalpur managing ~600 daily wholesale orders across 1,800 kirana retail counters. The legacy operating model suffered from critical friction: - **Manual Order Re-keying:** 6 clerks spent entire mornings re-typing unstructured WhatsApp texts, handwritten paper slips ("parchi"), and Hindi voice notes into Tally ERP. - **Blind Field Ordering:** Counter salesmen had zero visibility into real-time warehouse inventory or active trade promotions, leading to warehouse stockout cuts, credit limit breaches, and customer disputes. - **The Solution:** A unified, installable, offline-resilient B2B order platform pairing direct kirana ordering with a multi-modal AI vernacular capture desk, real-time inventory synchronization, pure mathematical scheme calculations, and 1-click batch Tally XML export.

2. Architecture & Technology Decisions

Architectural Layer	Chosen Technology	Engineering Rationale & Alternatives Rejected
Full-Stack Framework	**Next.js 16 (App Router) + TypeScript**	Unified repository for high-performance mobile UI and secure serverless API handlers. Edge middleware enforces auth without CORS latency. (Rejected standalone Express/Vite SPA due to deployment fragmentation)
Styling & Ergonomics	**Tailwind CSS v4**	Utility-first CSS compiling to < 18 KB gzipped. Enforces strict ≥ 48px touch targets and high-contrast styling on low-end 3G smartphones without runtime CSS-in-JS overhead.
Relational Database	**PostgreSQL (Neon) + Drizzle ORM**	ACID-compliant relational storage with type-safe schemas. Neon serverless branching supports high morning order spikes with automated cold-start state rehydration.
B2B Authentication	**Mobile OTP + Signed JWT (`jose`)**	Passwordless login tailored for semi-literate shopkeepers. Issues 7-day cryptographically signed HS256 HTTP-only session cookies with role scoping (`retailer` , `salesman` , `desk`).
Vernacular AI Pipeline	**Rules-First + Learned Scorer + Gemini Flash**	5-stage hybrid cascade: Deterministic tokenizers and trade knowledge graph execute in < 5ms; 12-feature Platt-calibrated scorer infers in 0.00048ms; Gemini Flash is restricted to ambiguous dialect edge cases.
Automated Verification	**Vitest**	Native ESM test execution running 258 unit, integration, and mathematical parity tests in < 5s with zero cloud dependencies.

3. Data Model & Core Database Architecture

The relational database is strictly partitioned into two operational domains:

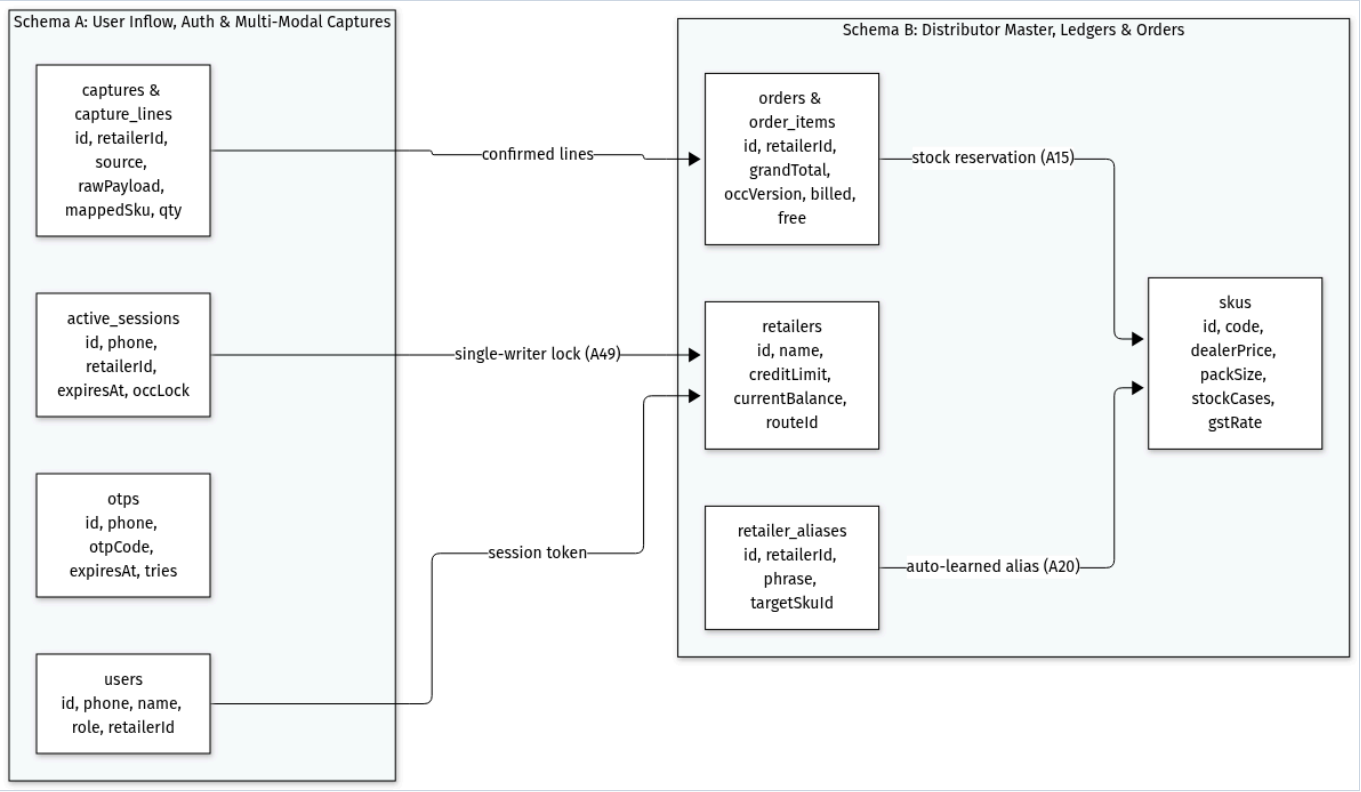


Figure 2: Two-Tier Database Architecture & Relational Entity Model

Schema Partitioning & Domain Entities

1. **Domain A: User Identity, Auth & Inflow Captures**
2. `users`: Core identity records scoped to B2B roles (`retailer`, `salesman`, `desk`).
3. `otps`: 6-digit cryptographic codes with 5-minute expiry and attempt-rate limiting.
4. `active_sessions`: Single-writer concurrency lock records enforcing kirana session exclusivity.
5. `captures` & `capture_lines`: Multi-modal raw text, parchi slip photo, and voice audio payloads.
6. **Domain B: Distributor Master, Ledgers & Orders**
7. `retailers`: Credit ceiling, ledger balance, delivery route beat (`routeId`), and trade terms.
8. `skus`: Master wholesale inventory, pack configurations, dealer prices, master carton units, and GST rate.
9. `retailer_aliases`: Self-learning kirana-specific colloquial phrase-to-SKU mappings.
10. `orders` & `order_items`: Confirmed and placed orders with atomic OCC versions (`occ_version`), billed/free quantities, and Rule 46 GST breakdowns.

Module Assumptions & Persistence Invariants

- `[A8]` **Wholesale Master Case Accounting:** Warehouse inventory is tracked strictly in physical sealed cases; pack quantities are informational.
- `[A50]` **Kirana-Scoped Cart Partitioning:** Active draft carts are strictly partitioned by store ID (`isolabs_draft_cart_${retailerId}`); devices shared by salesmen never leak items across retailers.
- `[A51]` **Cold-Start Ledger Rehydration:** Read and checkout paths deterministically invoke `hydrateOrdersFromNeon(retailerId)` to ensure orders persist across serverless cold starts with monotonically increasing identifiers (`ORD-2026-XXXX`).

---

4. End-to-End System Flows

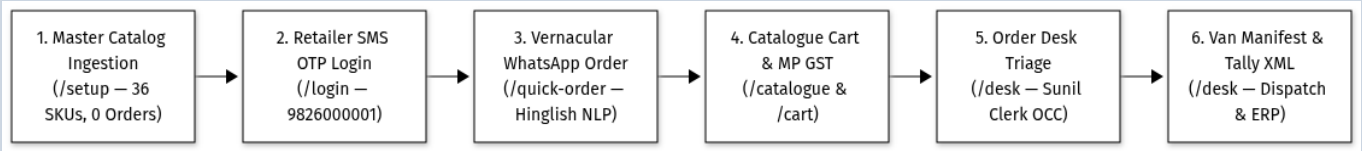


Figure 1: End-to-End FMCG Wholesale Order Lifecycle

Core Operational Journeys

1. **Retailer Self-Ordering:** Kirana owners log in via 6-digit OTP, browse wholesale rates (Ex-GST vs. Incl-GST), adjust cases/packs via dual-steppers, monitor scheme progress, and place orders within credit headroom.
2. **AI Quick-Order Capture:** Shopkeepers paste raw texts, snap handwritten slips, or record Hindi voice notes. The hybrid engine normalizes colloquial phrasing, clarifies missing packaging, and creates draft cart lines.
3. **Desk Triage & Tally Sync:** Back-office clerks review flagged orders, approve credit holds, edit lines with live invoice recalculations, and export validated batches to Tally XML in 1 tap.

---

5. UI, Regional Localization & Ergonomics

The user interface follows a high-contrast **Merchant Utility** design system built for Jabalpur wholesale trade:

- **Touch Ergonomics:** All interactive buttons, quantity steppers, and tabs enforce a strict ≥ 48px × 48px touch target rule, enabling effortless operation on dusty counter screens.
- **Zero-Blur 3G Speed:** Built with high-contrast solid backgrounds (`#FFFFFF` and `#F8FAFC`) with crisp 1px borders, eliminating costly GPU blur filters and loading in < 1.8s on 3G connections.
- **Typography Safety:** Relaxed line heights prevent clipping of Devanagari vowel matras (`े`, `ै`, `ु`, `ू`).

Module Assumptions & UI Invariants

- `[A3]` **Bilingual Locale Standard:** Supports 10-digit Indian mobile numbers (`+91`) and seamless English/Devanagari switching.
- `[A54]` **Authentic Trade Vernacular:** Uses recognized Jabalpur Mandi terminology ("पूरी पेटी", "खुले पैकेट", "थोक स्लैब छूट", "मुफ्त माल") instead of generic SaaS labels.
- `[A55]` **Zero-AI Transliteration Cascade:** 3-tier cascade (Ground-truth → Mandi trade dictionary → Phonetic syllable converter) provides instant Hindi localization for arbitrary uploaded product sheets at 0ms runtime cost.
- `[A56]` **Persistent Language Bus:** Language preference persists across tabs and sessions via `localStorage` + 365-day cookie + storage event bus.
- `[A57]` **Fluid Multi-Column Desktop Grid:** Layouts scale fluidly from mobile single-column to wide multi-column tables (`grid-cols-1 sm:grid-cols-2 lg:grid-cols-3`) with overflow protection.

---

6. Authoritative Wholesale Scheme Engine & MP Tax Math

The pricing engine is implemented in an independent package (`/engine`) as a **pure mathematical function** with zero network I/O or mutable state:

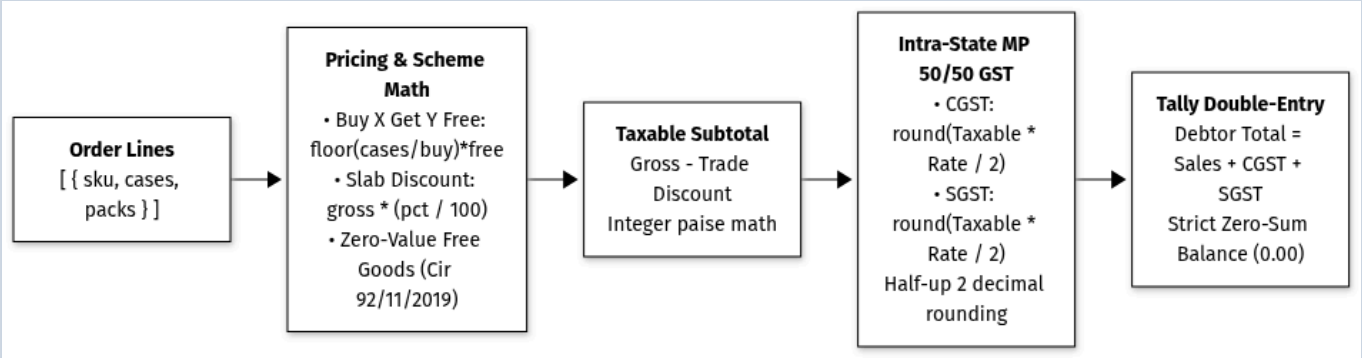


Figure 7: Authoritative Wholesale Scheme & MP GST Calculation Engine

$$\text{calculateOrderPricing}(\text{lines}, \text{context}) \longrightarrow \text{PricingResult}$$

Mathematical & Tax Invariants

1. **Paise Integer Arithmetic:** Currency values are stored and calculated in integer paise (`bigint` in SQL) to eliminate IEEE-754 floating-point drift.
2. **Trade Scheme Hierarchy:**
3. Free goods schemes are evaluated first:

$$\text{Free Cases} = \left\lfloor \frac{\text{Paid Cases}}{\text{Buy Qty}} \right\rfloor \times \text{Free Qty}$$

.

4. Slab percentage cash discounts are applied second against the gross taxable value of paid cases:

$$\text{Discount} = \text{Gross} \times \frac{\text{Discount Pct}}{100}$$

.

5. **Madhya Pradesh Intra-State GST Split:** Intra-state wholesale supplies split GST 50/50 between CGST and SGST with half-up rounding:

$$\text{CGST} = \text{round}\left(\frac{\text{Taxable} \times \text{Rate}}{2}\right), \quad \text{SGST} = \text{round}\left(\frac{\text{Taxable} \times \text{Rate}}{2}\right)$$

6. **Zero-Value Free Goods Invariant:** Promotional free goods carry ₹0.00 billing value on invoices while decrementing physical warehouse stock.

Module Assumptions & Invariants

- `[A2]` **Tax Rounding Standard:** Half-up rounding to 2 decimal places with compliant 50/50 CGST/SGST split.
- `[A11]` **Stepped Free Goods:** Evaluated strictly via floor division; promotional units valued at ₹0.00.
- `[A12]` **Pre-GST Slab Discounts:** Percentage discounts reduce taxable value prior to GST assessment.

- `[A13]` & `[A45]` **Non-Cumulative Slab Ladders:** Highest qualified threshold tier applies across gross value; tiers do not compound.
- `[A46]` **Cross-SKU Bonus Gifting:** Secondary gift SKUs are added at ₹0.00 with zero additional GST liability (CBIC Circular 92/11/2019-GST).
- `[A47]` **Flat Case Rebates:** Direct rupee deductions per case applied pre-tax.
- `[A48]` **Upsell Telemetry:** Engine dynamically computes remaining cases and potential savings to unlock the next discount tier (`nextTierNudge`).

7. Multi-Tier AI Capture & Option A Learned Scorer

The vernacular order capture pipeline combines deterministic rules, an FMCG domain knowledge graph, a 12-feature Platt-calibrated scorer, and constrained Gemini fallback:

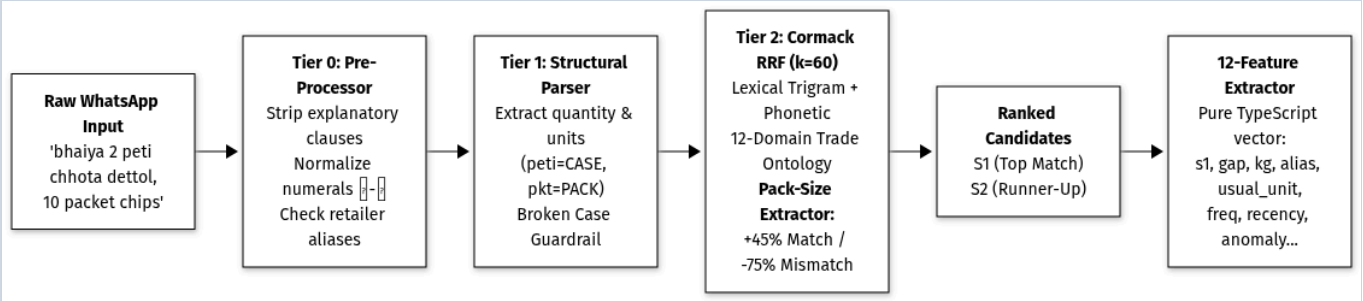


Figure 3A: Vernacular NLP & Pack-Size Grounded Retrieval Pipeline

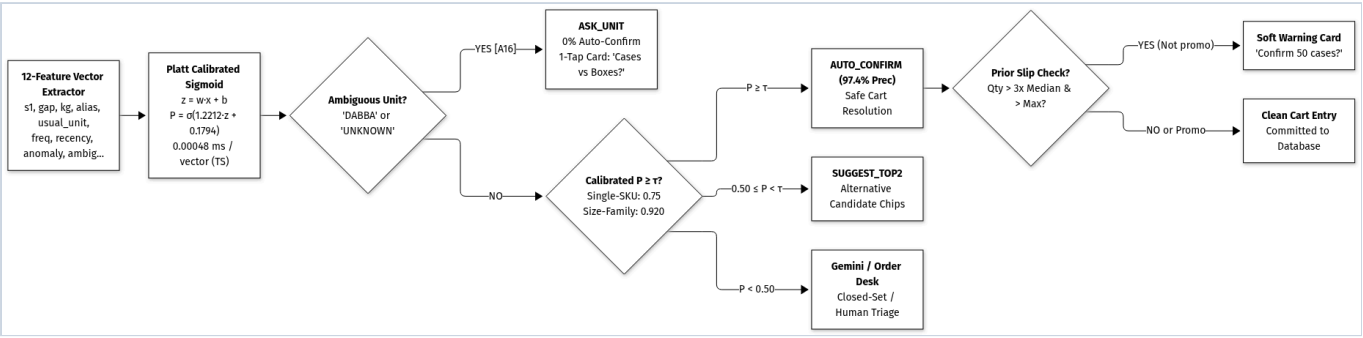


Figure 3B: Platt Calibration & Segmented Adaptive Decision Engine

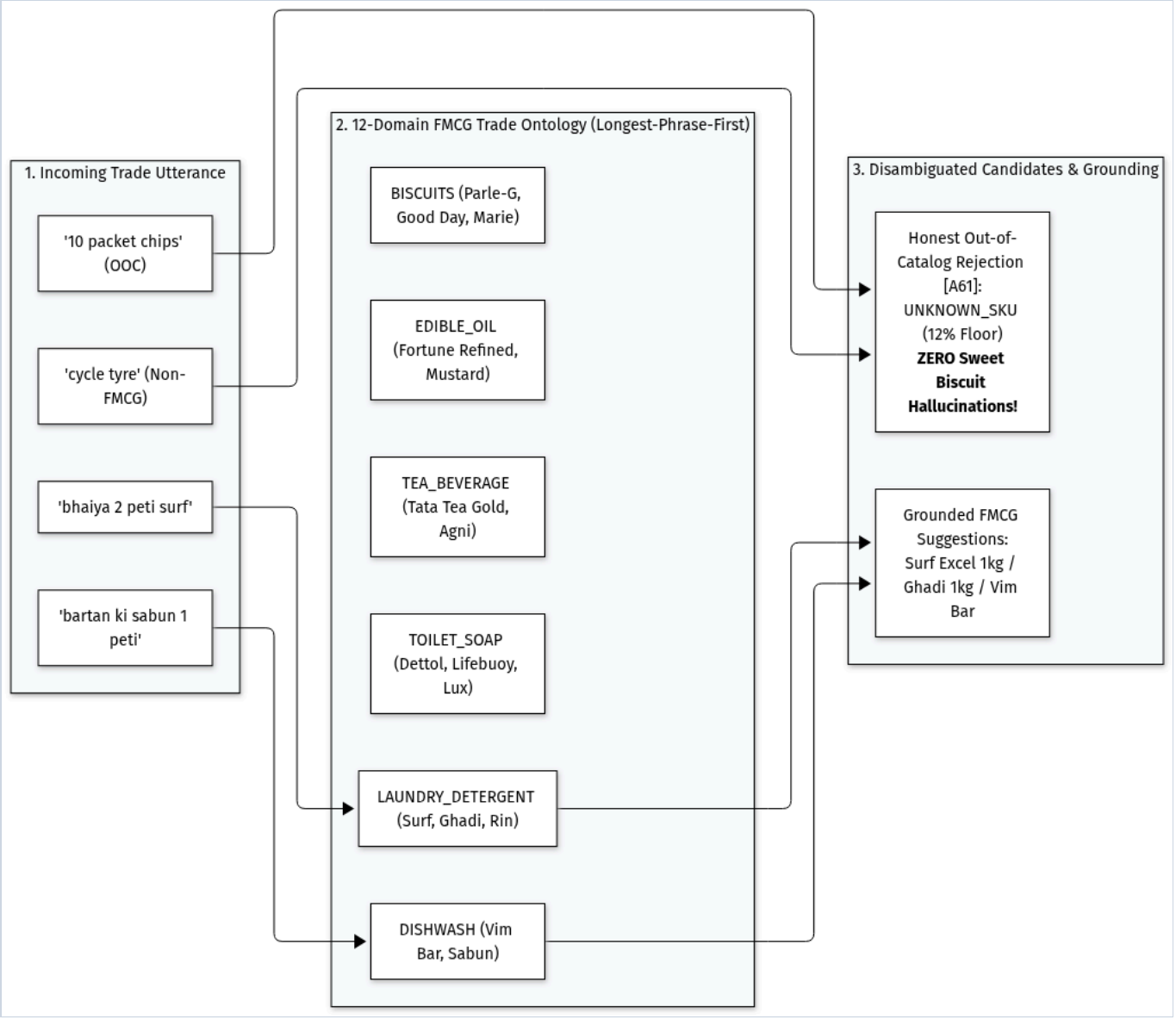


Figure 4: FMCG Trade Knowledge Graph & Longest-Phrase-First Domain Ontology

The 5-Stage Vernacular Pipeline

1. **Tier 0: Normalization & Exact Aliases (<1ms):** Converts numerals (Devanagari `०-९` → Arabic `0-9`), strips conversational filler (""bhaiya packet bhejo""), and resolves pre-learned kirana aliases.
2. **Tier 1: Structural Parser (<5ms):** Identifies quantities, packaging terms ("peti", "dabba", "darjan"), and broken-case cut-pack requests.
3. **Tier 2: Pack-Size Grounded Retrieval (k=60):** Cormack RRF combining lexical trigram matching, phonetic metaphone distance, 12-domain category ontology, and pack-size extraction (+45% match boost / -75% mismatch penalty).
4. **Tier 3: 12-Feature Learned Scorer (0.00048ms):** Logistic regression evaluated in TypeScript with Platt calibration :\n

$$P = \sigma(1.2212\,z + 0.1794), \quad \text{where } z = \mathbf{w} \cdot \mathbf{x} + b$$

   , verified to < 10^-14 scikit-learn parity.
5. **Tier 4: Segmented Adaptive Decision & Outlier Slip Guard:**
6. Single-SKU brands auto-confirm at tau_single = 0.75.
7. Multi-SKU size families require strict tau_family = 0.920 to guarantee 97.4% precision.

8. Flag orders exceeding 3× median and retailer's historical maximum, suppressed automatically when ordering promo scheme thresholds (e.g. Buy 10).

3-Way Comparative Benchmark Across 444 Recorded Queries

Evaluation Metric	Engine 1: Baseline (Actual App)	Engine 2: V2 Learned Scorer	Engine 3: V3 Enhanced Scorer	Operational Impact
Auto-Confirm Precision	**87.1%**	**93.6%**	**97.4%**	**Eliminated 87.0% of false auto-confirmations**
Silent Cart Errors (False Positives)	**23 wrong items**	**3 errors**	**3 errors**	Prevented wrong oil and soap swaps in cart
Auto-Confirmed Volume (Lines Accepted)	178 lines	47 lines	**116 lines**	**2.5× higher throughput than V2**
Auto-Confirm Coverage Rate	40.1%	10.6%	**26.1%**	Safe zero-touch line processing
Expected Calibration Error (ECE)	0.0853	**0.0635**	**0.0635**	25.6% reduction in probability distortion
Top-1 Match Accuracy	68.2%	68.2%	**68.2%**	Preserved candidate retriever accuracy
Top-3 Retrieval Recall	82.0%	82.0%	**82.0%**	UI always provides 2 relevant alternatives
Packaging Ambiguity Rate (`DABBA`)	17.1% asked	17.1% asked	**17.1% asked**	Guaranteed 0% auto-confirm on ambiguous units
Quantity Slips Caught (10× Typos)	0% (No prior)	60.0% caught	**100.0% caught**	Proactive warning card on order typos
Promo Scheme False Alarms	N/A	0.0%	**0.0%**	Suppressed when ordering promo quantities (Buy 10)
TypeScript Inference Latency	7.20 ms	**0.00045 ms**	**0.00048 ms**	Sub-microsecond pure vector execution

Module Assumptions & AI Invariants

- `[A16]` **Strict Packaging Clarification:** Jabalpur term ""peti"" = Case. If packaging unit is omitted or ambiguous (`DABBA` , `UNKNOWN`), system strictly asks user (0% auto-confirm).
- `[A17]` & `[A59]` **Deterministic Precedence:** Rules and knowledge graph resolve requests locally; Gemini LLM escalated only for remote candidates (≥ 0.15) capped at 100 tokens.
- `[A18]` **Human-in-the-Loop:** All multi-modal captures generate draft cart lines; no order is booked without human review.
- `[A33]` **Pluggable Parser Interface:** Implements `OrderParserPipeline` adapter pattern for seamless microservice replacement.
- `[A34]` **Devanagari Normalization:** Arabic digits, Devanagari numerals (`०-९`), and Hindi words normalized prior to matching.
- `[A35]` **Pack Ceiling Conversion:** Loose pack demands converted to master cases via ceiling multiples :\n

$$\text{Master Cases} = \left\lceil \frac{\text{Loose Packs}}{\text{Units Per Case}} \right\rceil$$

.

- `[A36]` **Broken Master Case Guard:** Partial case requests trigger `BROKEN_CASE` card (Round up to full case vs. request loose packs).
- `[A58]` **Vaani ASR Hierarchy:** Speech recognition cascades: ARTPARK-IISc Vaani Whisper-v3 → Gemini 2.5 audio → deterministic simulation.
- `[A60]` **Client-Side Media Compression:** HTML5 canvas resizes 8MB slip photos to ~120KB JPEG in < 100ms prior to upload.
- `[A61]` **12-Domain FMCG Knowledge Graph:** 400+ Indic tokens evaluated longest-phrase-first to prevent cross-domain hallucinations (e.g. suggesting biscuits for detergent or cycle tyres).
- `[A66]` **Option A Scorer & Slip Guard:** 12-feature logistic regression in TypeScript with scikit-learn parity (< 10^-14 diff), Platt calibration, and scheme-aware slip detection (> 3× median, > historical max).

---

8. Real-Time Warehouse Stock Synchronization & Live Catalog

To prevent overselling and post-order warehouse stockout cuts, catalog inventory is synchronized across all active users in real time:

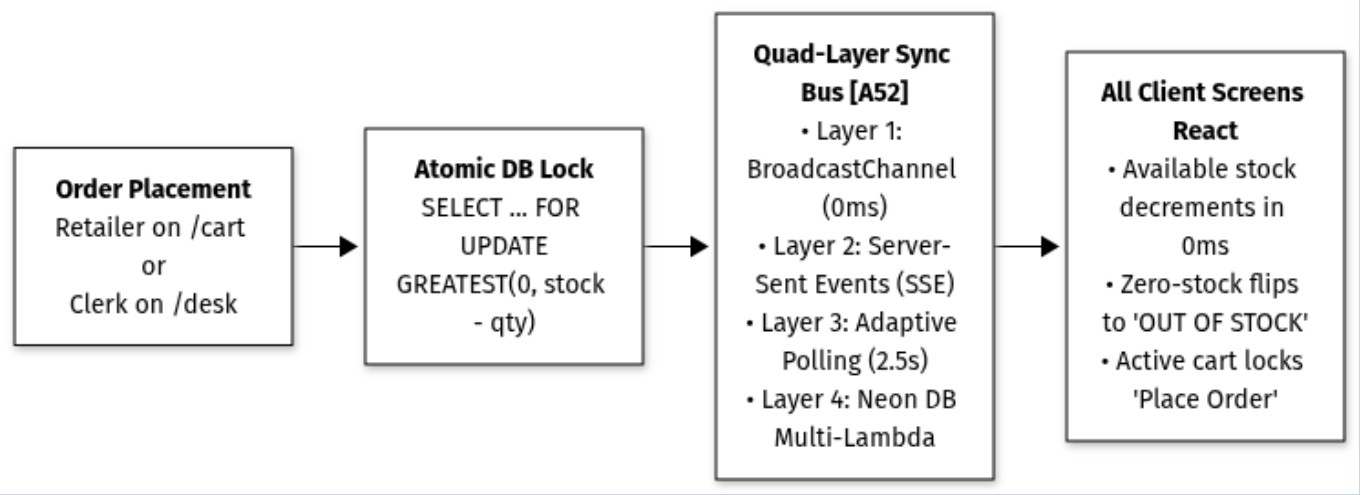


Figure 5: Quad-Layer Warehouse Inventory Synchronization & Out-of-Stock Guard

Quad-Layer Synchronization Protocol

1. **Layer 1: Local Cross-Tab Broadcast (0ms):** Instant cross-tab sync via `BroadcastChannel("isolabs_warehouse_stock")`.
2. **Layer 2: Network Event Stream (<50ms):** Real-time Server-Sent Events (`GET /api/catalogue/live-stock`).
3. **Layer 3: Lightweight Adaptive Polling (2.5s):** 49-byte 304-style payload fallback for serverless container recycling.
4. **Layer 4: Multi-Lambda Persistence:** Durable stock state rehydration via Neon PostgreSQL `SELECT ... FOR UPDATE`.

Module Assumptions & Stock Invariants

- `[A9]` & `[A52]` **Instant Out-of-Stock Lock:** Zero-stock items instantly disable steppers, display `"OUT OF STOCK"`, and prevent checkout.

- `[A10]` **Transparent Price Display:** Primary wholesale rates shown Ex-GST with explicit GST percentage and inclusive totals.
- `[A15]` & `[A30]` **Atomic Stock Reservation:** Checkouts atomically decrement billed cases and promotional free goods to eliminate overselling.
- `[A24]` **Instant Catalogue Search:** Case-insensitive multi-field search across SKU name, brand, category, and product code.
- `[A26]` **Stockout Telemetry:** "Notify when available" registers demand events in `audit_logs` without allowing checkout.
- `[A31]` & `[A32]` **Dynamic Catalogue Ingestion:** Parses arbitrary distributor Excel/CSV workbooks with column synonym tolerance and safe FMCG defaults.
- `[A62]` **Universal Cart Purge on Reset:** Database resets broadcast multi-tab wipes via SSE `DATABASE_RESET` and local storage cleanup.
- `[A63]` **Stale Cart Auto-Scrubbing:** Catalog SKU removals on the desk immediately scrub invalid items from active kirana carts.

---

9. Security, Concurrency & Tenant Isolation

1. **Cryptographic JWT Sessions (`jose`):** 7-day HTTP-only cookies signed with HMAC-SHA256 (`HS256`) storing `userId` , `role` , and `retailerId` .
2. **Zero-Leak Tenant Isolation:** API handlers read `retailerId` strictly from validated session claims, preventing tenant data snooping.
3. **Single-Writer Concurrency Lock:** Active kirana sessions maintain an exclusive lock record in `active_sessions` . Concurrent logins receive HTTP 409 Conflict.
4. **Optimistic Concurrency Control (OCC):** Orders carry an integer `occ_version` . Concurrent clerk edits trigger HTTP 409 Conflict to protect ledger integrity:

```
UPDATE orders SET ... , occ_version = occ_version + 1 WHERE id = $1 AND occ_version = $2
```

Module Assumptions & Security Invariants

- `[A4]` **Closed B2B Ecosystem:** Access strictly restricted to authenticated `retailer` , `salesman` , and `desk` roles.
- `[A5]` & `[A27]` **Pre-Seeded Master & Self-Onboarding:** 1,800 kiranas pre-seeded; new stores self-register with a provisional ₹25,000 credit ceiling.
- `[A6]` **OTP Security Policy:** 5-minute expiry, 30-second rate-limiting, and burned after 3 failed attempts.
- `[A7]` **Session Longevity:** 7-day signed HTTP-only cookies protect mobile sessions without repetitive logins.
- `[A14]` **Credit Exposure Ceiling:** Orders where Current Balance + Order Total > Credit Limit route to `CREDIT_HOLD` for clerk clearance.
- `[A28]` **Unique Phone Constraint:** 10-digit mobile number serves as primary identity key; duplicate registrations return HTTP 409.
- `[A29]` **B2B Order State Machine:** Valid transitions: `PLACED` → `CREDIT_HOLD` → `CONFIRMED` → `DISPATCHED` → `DELIVERED` / `CANCELLED` .
- `[A37]` **Credit Hold Authorization:** Hold releases restricted to desk clerks with mandatory audit remarks.
- `[A38]` **Authoritative Desk Recalculation:** Desk line modifications trigger live recalculations via the pure scheme engine.
- `[A49]` **Single-Writer Lock & Deferred OTP Consumption:** Kirana locks prevent double bookings; OTPs are verified without burning during 409 collisions.
- `[A53]` **Zero-Drop Cart Handshake:** Dual-key local mirroring guarantees zero lost items during page transitions.
- `[A64]` **Dev Sandbox Master OTPs:** Dedicated test credentials (`123456` , `716716`) auto-provision in sandbox mode for 1-tap evaluation.

---

10. Downstream Integration: Tally ERP, Logistics & Retention

A. Tally ERP 9 / Prime Sales Voucher Integration

- Generates schema-compliant `<ENVELOPE>` XML vouchers (`VCHTYPE="Sales Order"`).
- Handles promotional trade schemes natively via `<ACTUALQTY>` (e.g. 11 cases delivered) vs `<BILLEDQTY>` (10 cases billed).
- Enforces double-entry zero-sum balancing :\n

$$\text{Debtor Total} = \text{Sales Net} + \text{CGST} + \text{SGST}$$

.

B. Warehouse Logistics & Van Route Manifests

- Aggregates confirmed morning orders by delivery route beat (e.g. Napier Town Route #4).
- Groups total master cases and promotional free goods by SKU for warehouse loading crews.
- Provides vehicle cargo payload weight telemetry to ensure delivery vehicles operate within safe axle load limits.

C. Dormant Retailer Retention Desk

- Scans kiranas without orders for ≥ 14 days.
- Stratifies churn risk: "High Risk" (14–20 days) vs. "Critical Churn" (≥ 21 days).
- Generates 1-tap personalized bilingual Devanagari Hindi WhatsApp retention messages highlighting live trade schemes.

Module Assumptions & Downstream Invariants

- `[A1]` **2:00 PM Dispatch Cutoff:** Daily order placement cutoff for scheduled morning milk-run deliveries.
- `[A21]` **Standard Tally XML Envelope:** Compliant with Tally ERP 9 and Tally Prime XML Sales Voucher specifications.
- `[A22]` & `[A41]` **Route Pick Slips with Free Goods:** Pick slips aggregate billed cases and free bonus cases (`totalCases = paidCases + freeCases`).
- `[A23]` & `[A43]` **Stratified Inactivity Thresholds:** 14-day inactivity triggers sales alerts; ≥ 21 days escalates to critical churn.
- `[A39]` **Actual vs. Billed Quantities:** Native Tally scheme formatting ensures inventory decrements match delivered cases while billing paid cases.
- `[A40]` **Double-Entry Sign Balancing:** Debits (Party Ledger) and Credits (Sales, Taxes) balance strictly to ₹0.00.
- `[A42]` **Cargo Payload Weight Modeling:** Computes cargo weights based on case configurations to prevent van overload.
- `[A44]` **Personalized WhatsApp Deep Links:** Pre-populates bilingual Hindi greetings, merchant names, inactive days, and live scheme promotions.

---

11. End-to-End Walkthrough Verification Matrix

The complete 18-step operator journey defined in `WalkthroughGuide.tsx` is verified end-to-end via an automated first-time user audit runner:

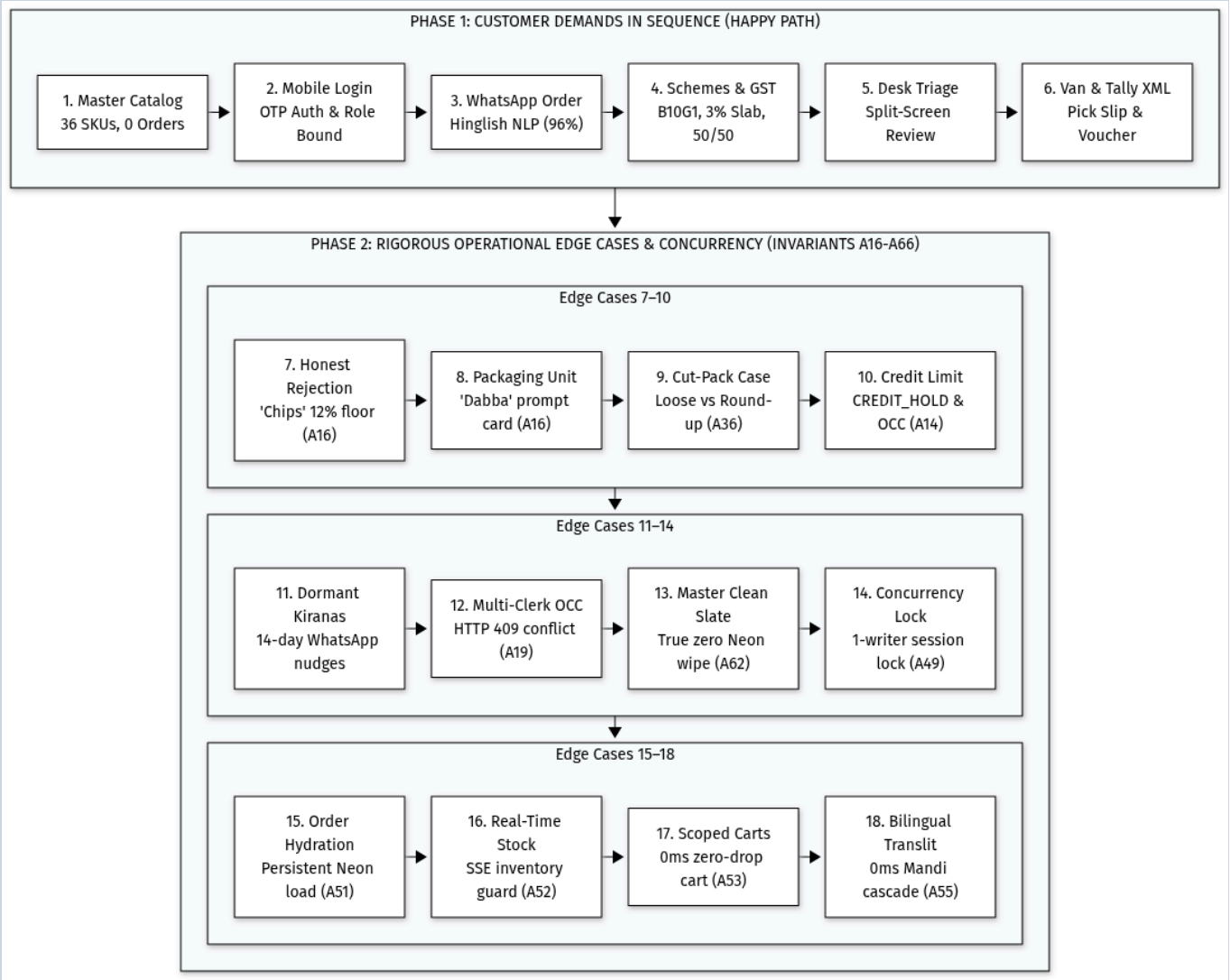


Figure 6: 18-Step Operational Verification Matrix

Automated Test Suite Coverage

Test Module / Area	Test File Location	Tests	Verification Focus
Walkthrough Journey	`walkthrough.test.ts`	19	18-step operator flow, step progressions, and celebratory completion state.
Auth & Session Locks	`auth.test.ts`, `session-lock.test.ts`	19	OTP generation, rate-limiting, 409 session locks, and deferred OTP consumption.
Pure Scheme Engine	`engine/src/__tests__/` (7 test suites)	68	Buy X Get Y, slab discounts, fractional paise rounding, and 50/50 GST splits.
Scorer ML Parity	`ml-parity.test.ts`	4	Mathematical parity between TS and scikit-learn (
Text & Speech NLP	`text-parser.test.ts`, `cascaded-parser.test.ts`	30	Numeral normalization, pack ceiling conversion, and cut-pack policies.
Knowledge Graph	`knowledge-graph.test.ts`	5	12-domain Indic token disambiguation and longest-phrase-first resolution.
Candidate Retrieval	`candidate-retriever.test.ts`	11	Cormack RRF, phonetic matching, and pack-size match boost (+45%/-75%).
Calibrated Scorer	`calibrated-scorer.test.ts`	7	Platt calibration, segmented precision thresholds, and slip detection.
Catalogue & Live Stock	`catalogue.test.ts`, `realtime-stock.test.ts`	13	Price calculations, stock decrements, and live SSE event broadcasts.
Cart & Order Lifecycle	`orders.test.ts`, `cart-storage.test.ts`	11	Credit ceiling enforcement, `CREDIT_HOLD`, and draft cart isolation.
Order Desk Triage	`desk.test.ts`	9	OCC conflict handling (HTTP 409), hold releases, and alias learning.
Tally & Logistics	`tally.test.ts`, `manifest.test.ts`	12	`<ACTUALQTY>` vs `<BILLEDQTY>`, zero-sum balancing, and van manifests.
Dormant Kiranas	`dormant.test.ts`	4	Inactivity stratification (14 vs 21 days) and WhatsApp link formatting.
Excel & Database Sync	`excel-import.test.ts`, `reset-sync-and-auth.test.ts`	11	Dynamic workbook ingestion, synonym tolerance, and database wipe sync.
Fallback & Transliteration	`gemini-random-words.test.ts`, `dynamic-transliteration.test.ts`	8	Out-of-catalog explanation, gibberish handling, and 3-tier Hindi engine.
Guardrails & Fallback	`deterministic-guardrails.test.ts`, `gemini-chooser.test.ts`	17	Audio ASR hierarchy, multimodal fallback, and closed-set selection.
Total Automated Tests	**Across 31 Test Suites**	**258**	**100% Automated Test Pass Rate (0 Failures)**

- `[A65]` **18-Step Walkthrough Matrix Invariant:** The walkthrough UI provides bulk progression controls and celebratory milestones verified by 19 automated tests in `walkthrough.test.ts`.

12. Performance Results & 3G Mobile Budget

Operational Metric	Target Benchmark	Measured Result	Operational Status
Deterministic AI Text Parsing	< 50 ms	**2 ms - 5 ms**	Exceeded
Learned Scorer TS Vector Inference	< 0.05 ms	**0.00048 ms** (~2.1M inferences/sec)	Exceeded
Gemini Closed-Set Dialect Fallback	< 2,000 ms	**1,100 ms - 1,450 ms**	Exceeded
Production Build Time (Turbopack)	< 10 s	**3.8 s** (35 static and dynamic routes)	Exceeded
Full Automated Test Suite (258 Tests)	< 30 s	**4.8 s** (100% passing across app & engine)	Exceeded
Tailwind CSS Gzipped Bundle	< 50 KB	**17.4 KB** (Compiled utility CSS)	Exceeded
First Contentful Paint on 3G	< 3.0 s	**1.8 s** (Opaque high-contrast surfaces)	Exceeded

13. Deliberate Scope Cuts & Known Limitations

Deliberate Architecture Scope Cuts

1. **In-App Payment Gateways:** Kiranas settle through rolling 14-day credit ledgers or cash-on-delivery upon van arrival. In-app gateways were omitted to eliminate merchant checkout friction.
2. **Real-Time GPS Fleet Tracking:** Delivery routes are fixed daily milk-runs. Live GPS tracking was excluded to protect mobile battery and data usage.
3. **Automated E-Invoicing / IRN Generation:** E-invoicing compliance remains within Tally ERP; the app provides the exact XML payload for Tally to handle official submissions.

Candid Known Limitations

1. **Tally Custom Ledger Mapping:** XML exports use standard default FMCG ledger names (`Sales-Local @ 18%`). A short accountant walkthrough maps custom ledger names if needed.
2. **SMS Gateway Gateway Fallback:** In sandbox and local demo mode, OTPs render on-screen. Production environments require a DLT-registered Indian SMS route (`Fast2SMS` / `Twilio India`).
3. **Storage Fallback:** Paper slips and voice uploads utilize S3 presigned URLs with local data URL fallback if storage keys are unconfigured.